

# Sorting Algorithms — Interview Revision Guide

Java Edition · Concepts, Code, Complexity & Interview Tactics

---

## 1. Quick Complexity Cheat Sheet

Memorize this table cold — it's the single most common "opening" interview question ("which sort would you use and why?").

| Algorithm      | Best          | Average       | Worst         | Space       | Stable | In-place |
|----------------|---------------|---------------|---------------|-------------|--------|----------|
| Bubble Sort    | $O(n)$        | $O(n^2)$      | $O(n^2)$      | $O(1)$      | Yes    | Yes      |
| Selection Sort | $O(n^2)$      | $O(n^2)$      | $O(n^2)$      | $O(1)$      | No     | Yes      |
| Insertion Sort | $O(n)$        | $O(n^2)$      | $O(n^2)$      | $O(1)$      | Yes    | Yes      |
| Merge Sort     | $O(n \log n)$ | $O(n \log n)$ | $O(n \log n)$ | $O(n)$      | Yes    | No       |
| Quick Sort     | $O(n \log n)$ | $O(n \log n)$ | $O(n^2)$      | $O(\log n)$ | No     | Yes      |
| Heap Sort      | $O(n \log n)$ | $O(n \log n)$ | $O(n \log n)$ | $O(1)$      | No     | Yes      |
| Counting Sort  | $O(n+k)$      | $O(n+k)$      | $O(n+k)$      | $O(n+k)$    | Yes    | No       |
| Radix Sort     | $O(nk)$       | $O(nk)$       | $O(nk)$       | $O(n+k)$    | Yes    | No       |
| Bucket Sort    | $O(n+k)$      | $O(n+k)$      | $O(n^2)$      | $O(n+k)$    | Yes    | No       |

## 2. Core Implementations (Java)

### Bubble Sort

```
static void bubbleSort(int[] a) {
    int n = a.length;
    for (int i = 0; i < n - 1; i++) {
        boolean swapped = false;
        for (int j = 0; j < n - 1 - i; j++) {
            if (a[j] > a[j + 1]) {
                int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
                swapped = true;
            }
        }
        if (!swapped) break; // early exit = best case O(n)
    }
}
```

**Interview tip:** Always mention the swapped flag optimization — interviewers specifically check if you know how to get the  $O(n)$  best case.

### Selection Sort

```
static void selectionSort(int[] a) {
    int n = a.length;
    for (int i = 0; i < n - 1; i++) {
        int minIdx = i;
        for (int j = i + 1; j < n; j++)
            if (a[j] < a[minIdx]) minIdx = j;
        int t = a[i]; a[i] = a[minIdx]; a[minIdx] = t;
    }
}
```

**Key point:** Always  $O(n^2)$  comparisons regardless of input — but minimizes number of swaps ( $O(n)$ ), useful when writes are expensive (e.g., flash memory).

### Insertion Sort

```
static void insertionSort(int[] a) {
    for (int i = 1; i < a.length; i++) {
        int key = a[i], j = i - 1;
        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;
    }
}
```

**Interview gold:** Mention it's the best choice for nearly sorted or small ( $n < \sim 10\text{-}20$ ) arrays — this is why Java's `Arrays.sort()` and Timsort switch to insertion sort for small sub-arrays.

## Merge Sort

```
static void mergeSort(int[] a, int l, int r) {
    if (l >= r) return;
    int mid = l + (r - l) / 2;
    mergeSort(a, l, mid);
    mergeSort(a, mid + 1, r);
    merge(a, l, mid, r);
}

static void merge(int[] a, int l, int mid, int r) {
    int[] temp = new int[r - l + 1];
    int i = l, j = mid + 1, k = 0;
    while (i <= mid && j <= r)
        temp[k++] = (a[i] <= a[j]) ? a[i++] : a[j++];
    while (i <= mid) temp[k++] = a[i++];
    while (j <= r) temp[k++] = a[j++];
    System.arraycopy(temp, 0, a, l, temp.length);
}
```

**Key point:** Guaranteed  $O(n \log n)$  even in worst case; stable; good for linked lists ( $O(1)$  extra space there) and external sorting. Downside:  $O(n)$  extra space for arrays.

## Quick Sort

```
static void quickSort(int[] a, int low, int high) {
    if (low < high) {
        int pi = partition(a, low, high);
        quickSort(a, low, pi - 1);
        quickSort(a, pi + 1, high);
    }
}

static int partition(int[] a, int low, int high) {
    int pivot = a[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (a[j] < pivot) {
            i++;
            int t = a[i]; a[i] = a[j]; a[j] = t;
        }
    }
    int t = a[i + 1]; a[i + 1] = a[high]; a[high] = t;
    return i + 1;
}
```

### Must-know follow-ups:

- Worst case  $O(n^2)$  happens on already-sorted input with a naive "last element" pivot → mitigate with random pivot or median-of-three.
- Not stable, but in-place ( $O(\log n)$  stack space).
- In practice faster than merge sort due to cache locality and smaller constants.

## Heap Sort

```
static void heapSort(int[] a) {
    int n = a.length;
    for (int i = n / 2 - 1; i >= 0; i--) heapify(a, n, i);
    for (int i = n - 1; i > 0; i--) {
        int t = a[0]; a[0] = a[i]; a[i] = t;
        heapify(a, i, 0);
    }
}

static void heapify(int[] a, int n, int i) {
    int largest = i, l = 2 * i + 1, r = 2 * i + 2;
    if (l < n && a[l] > a[largest]) largest = l;
    if (r < n && a[r] > a[largest]) largest = r;
    if (largest != i) {
        int t = a[i]; a[i] = a[largest]; a[largest] = t;
        heapify(a, n, largest);
    }
}
```

**Key point:** Guaranteed  $O(n \log n)$ ,  $O(1)$  space — best worst-case space/time tradeoff of the comparison sorts, but not stable and worse cache performance than quicksort.

## Counting Sort (non-comparison)

```
static void countingSort(int[] a, int maxVal) {
    int[] count = new int[maxVal + 1];
    for (int x : a) count[x]++;
    int idx = 0;
    for (int v = 0; v <= maxVal; v++)
        while (count[v]-- > 0) a[idx++] = v;
}
```

**Key point:**  $O(n+k)$  — beats the  $O(n \log n)$  comparison lower bound because it's not comparison-based. Only works for small integer ranges.

### 3. The "Why" Questions Interviewers Love

Q: Why can't comparison-based sorts beat  $O(n \log n)$ ?

A: With  $n$  elements there are  $n!$  possible orderings; a comparison-based algorithm is a decision tree where each comparison has 2 outcomes, so you need at least  $\log_2(n!) \approx n \log n$  comparisons to distinguish all orderings.

Q: Why does Java's `Collections.sort()` / `Arrays.sort(Object[])` use Merge Sort (Timsort) but `Arrays.sort(int[])` use Dual-Pivot Quicksort?

A: Objects need stability (equal elements must keep relative order, e.g., sorting by one field after another) — Timsort is stable. Primitives have no identity/equality concerns beyond value, so Quicksort's speed and  $O(1)$ -ish space wins; stability is irrelevant since two 5s are indistinguishable.

Q: What is Timsort?

A: A hybrid of merge sort and insertion sort. It finds/creates "runs" (already-ordered subsequences), sorts small runs with insertion sort, then merges runs using merge sort logic. Real-world data is often partially sorted, so this is very fast in practice —  $O(n)$  best case,  $O(n \log n)$  worst case.

Q: When would you pick Quick Sort over Merge Sort?

A: When average-case speed and low memory matter more than worst-case guarantees or stability (e.g., sorting primitives in memory). Pick Merge Sort when you need stability, guaranteed  $O(n \log n)$ , or are sorting linked lists / external (disk-based) data.

Q: How do you sort in  $O(n)$  time?

A: Only possible with non-comparison sorts (counting/radix/bucket) and only when you can exploit structure in the data (bounded integer range, fixed digit count, uniform distribution).

## 4. Classic Coding Problems (Sorting-Adjacent)

These get asked constantly — know the approach, not just the sort itself:

- **Sort an array of 0s, 1s, 2s (Dutch National Flag)** — single pass, three pointers (low, mid, high),  $O(n)$ ,  $O(1)$  space.
- **Kth largest/smallest element** — Quickselect (partition-based), average  $O(n)$ ; or a heap of size  $k$ ,  $O(n \log k)$ .
- **Merge two sorted arrays in-place** — start filling from the back to avoid overwriting.
- **Sort a nearly sorted array (each element  $\leq k$  away from sorted position)** — min-heap of size  $k+1$ ,  $O(n \log k)$ .
- **Count inversions in an array** — modified merge sort,  $O(n \log n)$ .
- **Find median of a data stream** — two heaps (max-heap for lower half, min-heap for upper half).
- **Sort a linked list** — merge sort ( $O(1)$  extra space possible, unlike arrays), since random access for quicksort/heapsort is poor on linked lists.
- **Sort strings by frequency / custom comparator** — know how to write `Comparator.comparing()` chains in Java.

### Example: custom comparator

```
Arrays.sort(people, (p1, p2) -> p1.age != p2.age  
    ? p1.age - p2.age  
    : p1.name.compareTo(p2.name));
```

## 5. Interview Tactics & Tips

- **Always ask clarifying questions first:** data size, integer range, memory constraints, stability requirement, mostly-sorted or random, duplicates allowed. This alone signals seniority.
- **State time/space complexity before coding**, not after — shows you're thinking, not guessing.
- **Trace through a small example by hand** (5-6 elements) before/after writing code — catches off-by-one bugs in partition/merge logic, which is where most candidates fail.
- **Know pivot selection strategies for Quick Sort** (first, last, random, median-of-three) — this is the #1 quicksort follow-up.
- **Be ready to code partition/merge from memory** — these two functions are asked far more often than full sort implementations.
- **Discuss stability with a concrete example:** e.g., sorting employee records first by department then by name — if the department sort isn't stable, the name ordering within a department can get scrambled.
- **Mention hybrid real-world algorithms** (Timsort, Introsort — quicksort that falls back to heapsort if recursion gets too deep, used in C++ `std::sort`) to show awareness beyond textbook algorithms.
- **If asked "optimize this further,"** the usual answers are: switch to insertion sort for small subarrays, use tail-call/iterative recursion to bound stack depth, or pick a non-comparison sort if data constraints allow it.